课程笔记

CS336 第 4 讲：Mixture of Experts 的机制、代价与工程现实

Codex 基于 Stanford CS336 2025 第 4 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 04 月 24 日
视频时长：1:22:04
视频链接：`https://www.youtube.com/watch?v=LPv1KfUXLCo`

目录

1 为什么 MoE 会在大模型时代重新变得重要 **2**

1.1 MoE 不是为了炫技，而是为了同 FLOP 下获得更高容量 2

1.2 为什么这几年 MoE 再次流行 2

1.3 但为什么 MoE 没有更早成为标准答案 3

1.4 本章小结 3

2 MoE 的基本结构：什么被替换，什么没有变 **4**

2.1 主干没变，通常只是把 MLP 换成 MoE 层 4

2.2 稀疏模型与 dense 模型的最重要区别是什么 4

2.3 本章小结 4

3 Routing：几乎所有实践最终都回到 top-k token-choice **4**

3.1 MoE 到底有哪些维度会变化 4

3.2 为什么 routing 讨论最后几乎总会回到 top-k 5

3.3 为什么 RL routing 没有成为主流 5

3.4 DeepSeek / Qwen 这类最近变体到底在改什么 5

3.5 本章小结 6

4 Balancing 与训练目标：为什么系统效率会反过来塑造优化目标 **6**

4.1 MoE 训练最大的系统性风险：专家使用不均 6

4.2 为什么课程说这些目标“有点 heuristic” 7

4.3 DeepSeek v3 的所谓 auxiliary-loss-free balancing 为什么值得注意 7

4.4 本章小结 7

5 DeepSeek 作为案例：现代大规模 MoE 不只是 top-k **8**

5.1 为什么课程最后选择用 DeepSeek 系列做收束 8

5.2 为什么现代 MoE 经常要和别的结构性改动一起出现 8

5.3 本章小结 8

6 总结与延伸 **9**

1 为什么 MoE 会在大模型时代重新变得重要

第 4 讲讨论的是 Mixture of Experts。它之所以重要，不是因为这是某种“新奇变体”，而是因为它抓住了大模型时代一个极其现实的问题：**如果我们继续想提高参数容量和模型能力，但又不想让每个 token 的计算成本同步爆炸，应该怎么办？**

在 dense Transformer 里，每个 token 都会经过同一套完整 FFN 层。参数越大，理论上每个 token 都要为这些参数买单。MoE 的直觉则是：既然不同 token 未必需要同样的专家能力，为什么不把一个大 FFN 替换成很多个大 FFN，再让一个 selector layer 只挑其中少数几个真正参与计算？

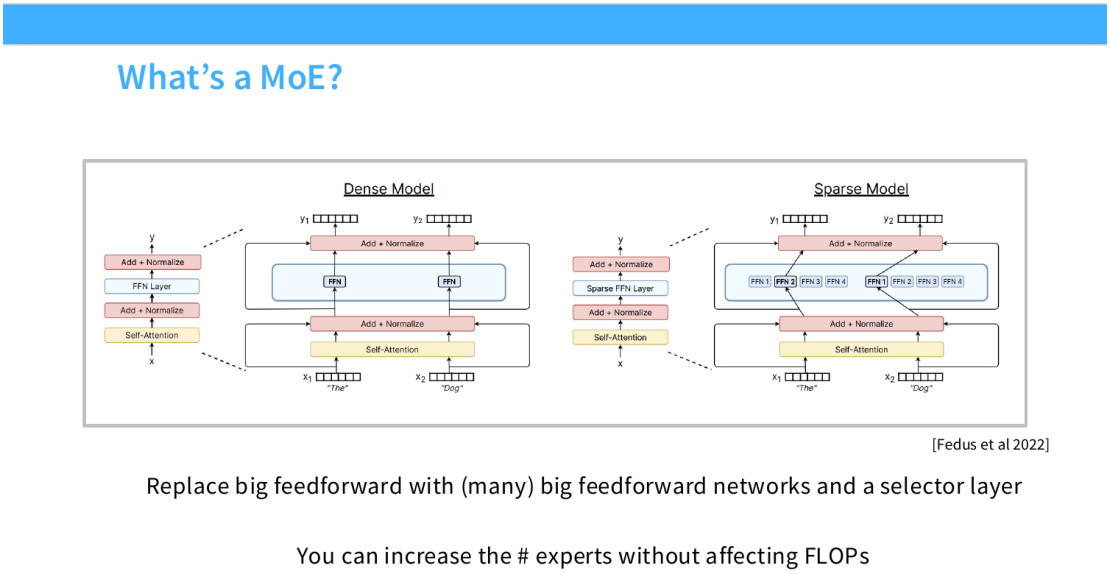

图 1: MoE 的基本思路：把 dense FFN 替换成一组专家网络，并对每个 token 做稀疏路由。

课程直接给出最关键的一句话：**你可以增加专家数，而不必让 FLOPs 按同样比例上升**。这就是 MoE 的吸引力所在。它试图把“总参数量”和“每个 token 实际激活的计算量”拆开。

1.1 MoE 不是为了炫技，而是为了同 FLOP 下获得更高容量

第 4 讲前半段反复强调一件事：MoE 的价值不是“参数量看起来很大”，而是“在相近 FLOP 预算下，更多参数容量往往能带来更好的性能”。这和第 3 讲的架构共识思路是连着的。架构设计不是追求造型花哨，而是为了在固定资源账本下，把能力往上推。

第 4 讲的核心判断

MoE 的本质不是免费午餐，而是一种稀疏计算策略：把“更多参数”变成“并非每个 token 都要完整激活的更多参数”，从而在相近训练或推理成本下交换更高模型容量。

1.2 为什么这几年 MoE 再次流行

课程给出的原因非常务实，而且基本可以分成四类。

第一，**same FLOP, more parameters often works better**。如果每步训练的计算量差不多，但总参数容量更大，那么模型可能用相同计算预算获得更好的表示能力。

第二，**训练速度可能更快**。在一些设置下，MoE 不只是“最终效果更好”，还可能更快达到相近质量。

第三，**它在开放模型里已经被反复验证**。课程直接指出，很多高性能开放模型都走向了 MoE，而不再是纯 dense 路线。

第四，**它天然适合更多设备并行**。因为专家本来就是分块的，这使得把不同专家放到不同设备上在概念上更自然。

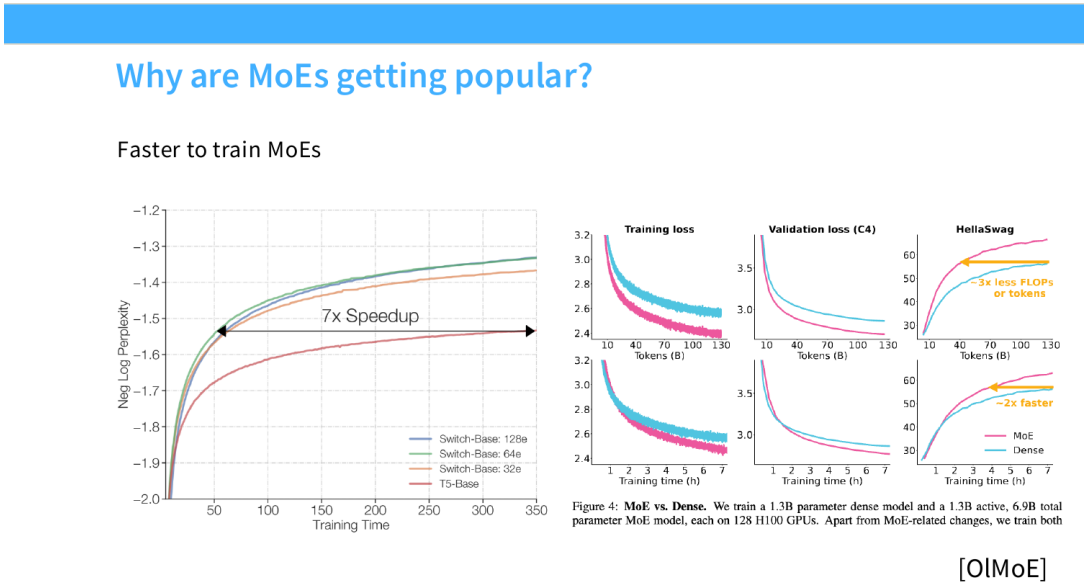

图 2: 课程用 OLMoE 等结果说明：MoE 的价值不仅仅是更大参数量，而是可能在 wall-clock 和样本效率上都更有优势。

1.3 但为什么 MoE 没有更早成为标准答案

课程也非常坦率地谈到了 MoE 的麻烦。它没有把 MoE 包装成“大家终于发现的正确道路”，而是指出它长期不够流行的两个主要原因：

- 基础设施复杂，尤其是在多机多卡环境下。
- 路由与 balancing 相关训练目标很多带有启发式色彩，而且有时不够稳定。

这两个问题一点都不小。dense 模型的好处之一，就是虽然贵，但结构相对规整；而 MoE 引入了显式稀疏路由、token 分配不均、跨设备通信和启发式损失，系统复杂度立刻上升。

为什么 MoE 不能被理解为“参数翻倍但 FLOPs 不变”

即使理论上只激活少数专家，真实系统里仍然会出现路由计算、token 重排、通信、负载不均和训练稳定性问题。这些开销不会让 MoE 失去价值，但它们足以让“稀疏 = 免费”这种说法失真。

1.4 本章小结

这部分的作用是把 MoE 放回第 2 讲和第 3 讲的共同框架里看。第 2 讲告诉我们要盯住资源账本，第 3 讲告诉我们要识别架构共识，而第 4 讲则把这两件事结合成一个更尖锐的问题：**能否用稀疏激活，把更大参数容量塞进相近预算里？**

2 MoE 的基本结构：什么被替换，什么没有变

2.1 主干没变，通常只是把 MLP 换成 MoE 层

第 4 讲对 MoE 的第一原则非常明确：绝大多数情况下，MoE 不是把整个 Transformer 彻底推翻，而是**主要替换掉原来的 FFN / MLP 子层**。也就是说，注意力结构、残差流、norm 等主干通常保持不变，变化集中发生在“token 进入哪一个专家前馈网络”这一层。

这种设计很有工程意义。因为如果你只替换 MLP，就能在不破坏整条主干结构的前提下，把稀疏化收益集中落在最吃参数的那部分组件上。课程提到，也有人做 attention-side 的 MoE，但那不是最常见路线。

2.2 稀疏模型与 dense 模型的最重要区别是什么

最值得抓住的区别只有一个：**在 dense 模型里，每个 token 都经过同一套前馈参数；在 MoE 里，每个 token 只经过被路由选中的少数专家**。

这意味着：

- 总参数量可以很大，因为专家很多。
- 单 token 计算量不必与总参数量同比上升，因为只激活 top-k 专家。
- 真正的关键问题从“参数规模”转移到了“路由是否合理，以及专家是否被充分利用”。

为什么课程几乎总把 MoE 和 FFN 绑定讨论

FFN 在大模型里通常是参数大户之一，把稀疏化集中放在这里，能最大化“更多参数容量但不过度增加每-token 成本”的收益。也正因为如此，MoE 最常见的形态就是“把 FFN 变成一组专家 FFN”。

2.3 本章小结

MoE 并不是一类完全不同的语言模型，而更像是“在 Transformer 主干内，对 FFN 进行稀疏化重构”的家族方法。理解这一点之后，后面关于 routing、expert size 和 balancing 的讨论就自然了：因为真正变化的，就集中在这几处。

3 Routing：几乎所有实践最终都回到 top-k token-choice

3.1 MoE 到底有哪些维度会变化

课程很早就把 MoE 的设计空间压缩成三大类：

- Routing function
- Expert sizes
- Training objectives

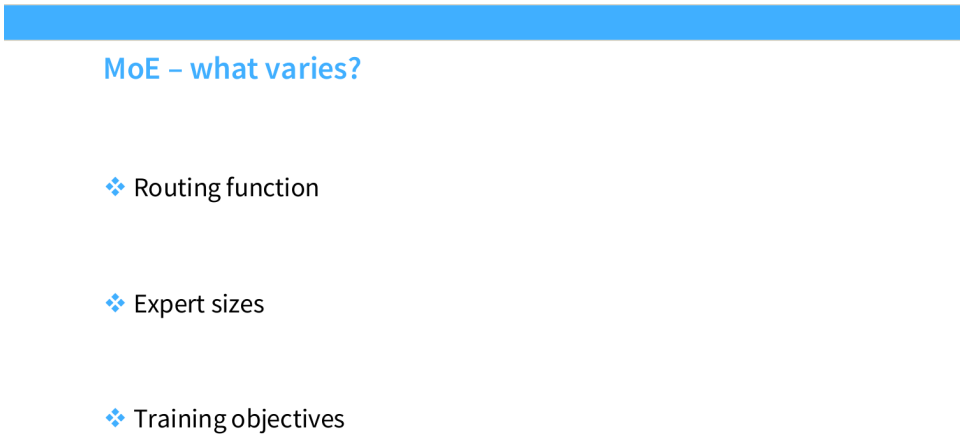

图 3: 课程用一页标题图把 MoE 变化空间压缩成三件事：路由、专家大小、训练目标。

这三类拆分非常有帮助，因为它避免把 MoE 讨论成“无数 paper 细节堆起来的黑箱”。几乎所有变化，最终都可以落回这三个问题：token 怎么选专家、专家本身多大、训练时如何让路由不塌。

3.2 为什么 routing 讨论最后几乎总会回到 top-k

课程回顾了几种 routing 思路：token-choice、expert-choice、全局优化、hash routing、RL、线性匹配等。看起来花样很多，但它马上指出一个非常现实的结论：**绝大多数主流系统最后还是选择了标准 token-choice top-k**。

原因并不神秘。top-k 足够简单，可微近似和实现路径都比较成熟，而且能在“效果、复杂度、系统友好性”之间达成很现实的平衡。Switch Transformer 用过 k=1；很多后续模型用 k=2；更晚近系统又探索了更大的 k，例如 Qwen、DBRX、DeepSeek 系列。

3.3 为什么 RL routing 没有成为主流

从理论直觉看，RL 似乎才是更“正确”的路由学习方案，因为 routing 本身就是一个离散决策问题。课程也承认，REINFORCE 一类办法是能工作的。但现实问题在于：**方差大、实现复杂，而且收益并没有大到足以碾压 top-k 启发式**。

于是工程世界最终选择了一条更粗糙却更可用的路线：直接用 top-k gating，再用各种 balancing 手段把系统维持在可训练状态。

为什么“理论更优”不一定赢过启发式

MoE 再次说明了一个大模型工程常识：如果一个方法虽然概念上更优雅，却大幅增加训练复杂度和不稳定性，那么实践里很可能会输给一个更朴素但更稳定、可扩展的启发式方案。

3.4 DeepSeek / Qwen 这类最近变体到底在改什么

课程指出，较新的 Chinese LMs 尤其 DeepSeek 和 Qwen，在 routing 之外又做了一个很重要的工程调整：**更细粒度的专家划分，加上少数始终开启的 shared experts**。

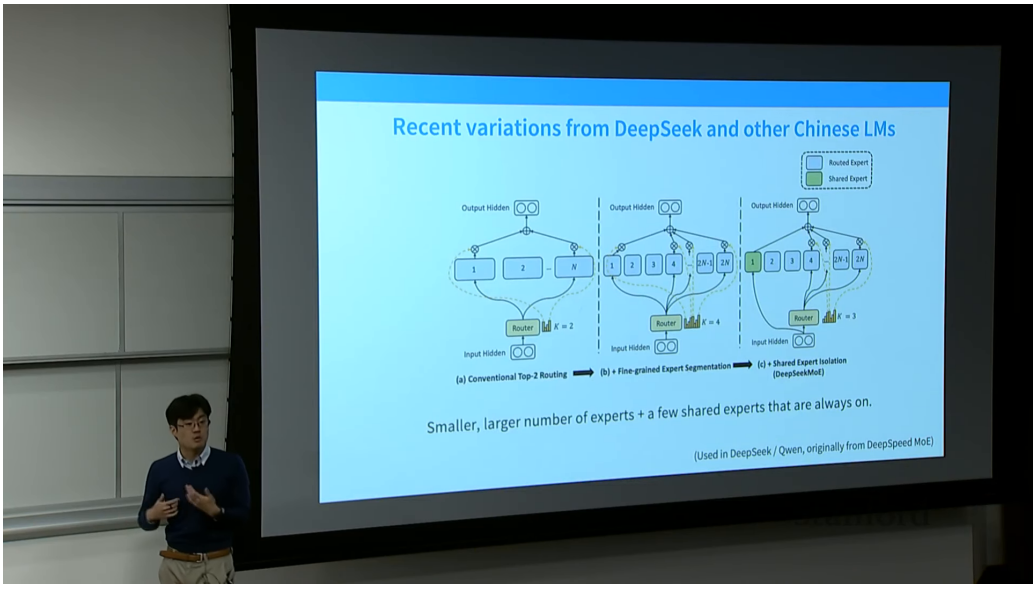

图 4: 课程展示了 shared experts 与 fine-grained experts 的组合思路：不是只靠更大 expert 数量，而是重新设计专家粒度与永远开启的公共通道。[1]

这个设计的直觉很强：shared experts 负责更通用的表示处理，routed experts 负责更具条件性的专门化能力。课程也提醒，相关论文 ablation 并非完全一致，有的工作认为 shared experts 明显有用，有的工作则认为收益更多来自 finer-grained experts 本身。这恰恰说明，MoE 的“正确答案”往往是数据、系统和实现细节共同作用的结果。

3.5 本章小结

Routing 这一节的真正收获是：虽然 MoE 论文很多，但实践层面的收敛比想象中明显。top-k token-choice 是当代主流默认值；shared experts 与 finer-grained experts 是近几年非常值得注意的变体；而更复杂的 routing 方法并没有自动成为赢家。

4 Balancing 与训练目标：为什么系统效率会反过来塑造优化目标

4.1 MoE 训练最大的系统性风险：专家使用不均

一旦引入稀疏路由，一个最直接的问题就出现了：**如果大部分 token 都被送往少数几个专家，其余专家几乎闲置，系统就既低效又难训**。于是 balancing 不再是锦上添花，而变成了 MoE 是否能稳定工作的核心条件。

课程在这里展示了负载均衡损失的直觉：频繁被选中的专家会被显式下调，从而鼓励 token 分配更均匀。你可以把这理解成一种面向系统效率的训练正则项。

[1] 视频画面时间区间：00:18:10–00:18:35。该图来自课程视频中的屏幕画面。

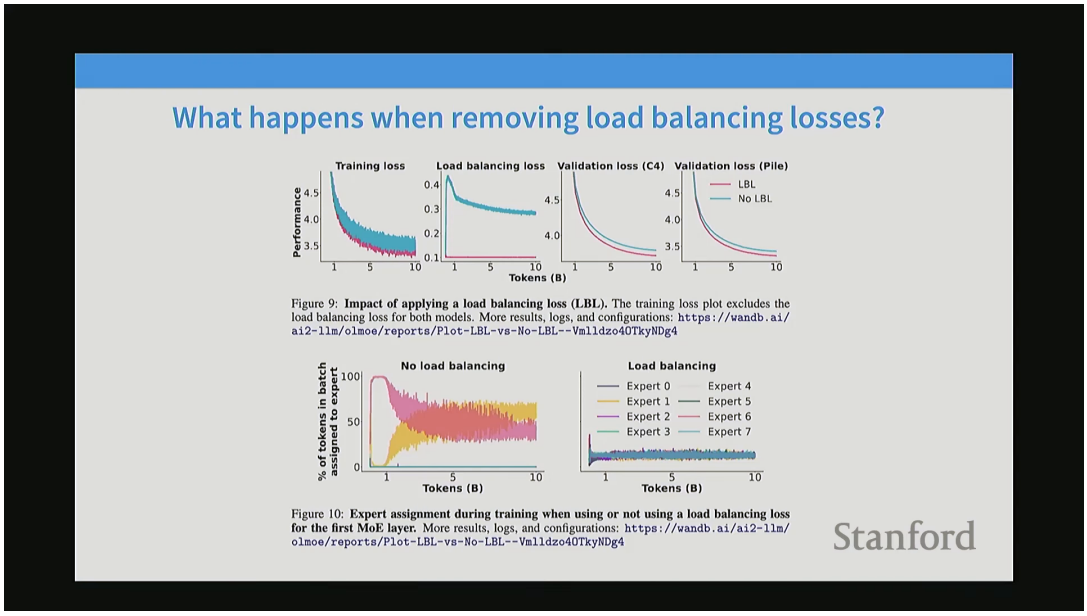

图 5: 课程直接展示“移除 load balancing loss 会怎样”的结果：性能和专家分配都会明显恶化。[2]

这张图的意义非常大，因为它说明了第 4 讲的一个核心思想：**MoE 的训练目标并不只是语言建模损失，它还必须对系统约束作出回应。**

4.2 为什么课程说这些目标“有点 heuristic”

课程对 balancing losses 的评价很坦率：它们很多时候就是 heuristic。不是说没用，相反它们经常非常有用；而是说它们未必来自一个优美、统一、被完全证明的理论框架。它们更像是为了解决“稀疏路由让系统塌掉”这个现实问题而逐步演化出来的工具箱。

4.3 DeepSeek v3 的所谓 auxiliary-loss-free balancing 为什么值得注意

课程还提到 DeepSeek v3 的一个有趣变体：通过 per-expert bias 的在线调节，让某些专家更可能被选中，从而在不完全依赖传统 auxiliary loss 的情况下改善负载均衡。课程也很谨慎地指出，这种方法并不是真的“完全没有 auxiliary 目标”，但它代表了 balancing 设计的一种新方向。

为什么 balancing 设计会持续演化

因为它同时面对两个目标：一是语言建模性能不能掉，二是专家利用必须足够均匀、系统吞吐不能太差。只要这两个目标之间还有张力，就会不断有新设计出现。

4.4 本章小结

在 MoE 里，训练目标第一次被非常显性地系统化了。它不仅服务于预测 token，也服务于“让专家被合理使用”。这使得 MoE 成为一个非常典型的例子：**系统效率要求会反向塑造优化目标本身。**

[2] 视频画面时间区间：00:37:30–00:37:55。该图来自课程视频中的屏幕画面。

5 DeepSeek 作为案例：现代大规模 MoE 不只是 top-k

5.1 为什么课程最后选择用 DeepSeek 系列做收束

课程结尾选择 DeepSeek v1 / v2 / v3 作为案例，不是因为它是唯一好的 MoE 系统，而是因为它把最近几年很多关键设计都放到了一个连续演化谱系里：fine-grained experts、shared experts、不同 balancing 机制，以及与 MoE 配套的其他系统设计。

Earlier MoE results from Chinese groups - DeepSeek

There's also some good recent ablation work on MoEs showing they're generally good

Metric	# Shot	Dense	Hash Layer	Switch
# Total Params	N/A	0.2B	2.0B	2.0B
# Activated Params	N/A	0.2B	0.2B	0.2B
FLOPs per 2K Tokens	N/A	2.9T	2.9T	2.9T
# Training Tokens	N/A	100B	100B	100B
Pile (Loss)	N/A	2.060	1.932	1.881
HellaSwag (Acc.)	0-shot	38.8	46.2	49.1
PIQA (Acc.)	0-shot	66.8	68.4	70.5
ARC-easy (Acc.)	0-shot	41.0	45.3	45.9
ARC-challenge (Acc.)	0-shot	26.0	28.2	30.2
RACE-middle (Acc.)	5-shot	38.8	38.8	43.6
RACE-high (Acc.)	5-shot	29.0	30.0	30.9
HumanEval (Pass@1)	0-shot	0.0	1.2	2.4
MBPP (Pass@1)	3-shot	0.2	0.6	0.4
TriviaQA (EM)	5-shot	4.9	6.5	8.9
NaturalQuestions (EM)	5-shot	1.4	1.4	2.5

图 6: DeepSeek 等近期结果说明，MoE 不只是一个理论可能性，而是在公开模型里已经被反复验证的现实路线。

DeepSeek 的价值在于，它把 MoE 从“用 top-k 稀疏化一下 FFN”推进成了更完整的系统工程：不仅要有路由，还要重新思考专家粒度、共享专家、通信平衡、推理缓存和训练稳定性。

5.2 为什么现代 MoE 经常要和别的结构性改动一起出现

课程最后还顺手提到了 MLA 和 MTP。它真正想说的是：**大规模 MoE 的收益常常要和别的系统性优化叠加才能完全兑现**。例如，若 KV cache 成本很高，仅靠 MoE 并不能自动解决推理瓶颈；于是像 MLA 这样的低维潜变量注意力设计才会变得重要。

这再次回到 CS336 的总主线：现代大模型设计几乎从来不是“只改一个模块”，而是多项决定同时围绕资源、稳定性和能力在做联合优化。

第 4 讲的压缩结论

MoE 真正让人兴奋的地方，不是“它是稀疏模型”，而是它证明了一个更一般的命题：在大模型时代，参数容量、训练成本和系统复杂度之间并不是一条单线关系，架构可以通过稀疏激活重新改写这三者的耦合方式。

5.3 本章小结

DeepSeek 这类案例的作用，是把整讲从抽象设计空间拉回真实系统。到这里你会发现，第 4 讲不只是讲 MoE 本身，更是在教你怎样评估一类看似复杂的工业系统设计：它解决了什么资源问题，

又引入了什么新复杂度。

6 总结与延伸

第 4 讲最重要的结论可以压成四句话。

第一，**MoE 的核心价值是把“总参数量”和“单 token 实际计算量”解耦**。这使得模型可以在相近 FLOP 预算下拥有更高总容量。

第二，**routing 设计虽然花样很多，但实践世界已经明显向 top-k token-choice 收敛**。这再次说明工程世界偏好“足够好且可扩展”的方案。

第三，**balancing 不是附加小技巧，而是 MoE 系统能否高效工作的一部分**。一旦专家分配失衡，MoE 的理论收益就会迅速被系统问题吞掉。

第四，**现代大规模 MoE 必须作为整套系统工程来看**。DeepSeek 这样的系统说明，真正有效的 MoE 往往不是“替换掉 FFN 就结束”，而是路由、平衡、通信、推理与其他结构优化共同联动的结果。

我对第 4 讲的最终提醒

不要把 MoE 理解成“只要稀疏化就能白捡性能”。MoE 的真正价值建立在一个前提上：你有能力把 routing、load balancing、通信和稳定性这些麻烦一起处理好。

从后续学习路径看，这一讲和后面的系统课、推理课联系极强。因为 MoE 虽然首先看起来是架构问题，但它几乎每一个收益点都直接依赖系统是否能承接；而它最麻烦的地方，恰恰也都在系统层爆发。